Mikroprocesorová technika

Prednáška č.5: Komunikačné rozhranie USART

prof. Ing. Ivan Virgala, PhD.

Technická univerzita v Košiciach
Strojnícka fakulta

Katedra priemyselnej automatizácie a mechatroniky
ARM-lab (Applied Robotics and Mechatronics Laboratory)



Sériová komunikácia a USART

Prenos bajtov medzi mikrokontrolérom a externým zariadením

Tematické vymedzenie

Obsah prednášky

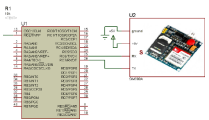
- princíp sériového asynchrónneho prenosu a dátového rámca,
- periféria `USART0` mikrokontroléra `ATmega328P`,
- nastavenie prenosovej rýchlosti a formátu rámca,
- odosielanie a príjem bajtov v jazyku C,
- znaky, reťazce, polia a ukazovatele potrebné pre textovú komunikáciu,
- blokujúci prístup, prerušenia a základná organizácia ovládača USART.

Použitá platforma

Arduino UNO R3 s mikrokontrolérom `ATmega328P`, programovanie v jazyku C bez Arduino frameworku a priamy prístup k registrom periférie.

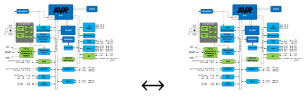
Príklady použitia USART

Snímač / modul ↔ mikrokontrolér



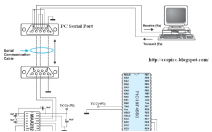
USART môže prepájať mikrokontrolér so snímačom alebo externým modulom, ktorý má sériové rozhranie – napr. merací, GNSS alebo komunikačný modul.

Mikrokontrolér ↔ mikrokontrolér



Dva riadiace uzly si môžu priamo vymieňať príkazy, stavové premenné alebo výsledky merania cez vodiče TX a RX.

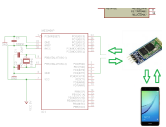
Počítač ↔ mikrokontrolér



USART sa používa na diagnostiku, servis a konfiguráciu cez terminál; podľa fyzického rozhrania môže byť potrebný USB–UART alebo RS-232 prevodník.

USART v komplexnejších zariadeniach

Bezdrôtový komunikačný modul



Mikrokontrolér môže cez USART komunikovať s Bluetooth, Wi-Fi, GNSS/GPS, GSM/LTE alebo iným komunikačným modulom.

Inteligentná domácnosť



USART môže lokálne prepájať riadiaci mikrokontrolér so snímačmi, ovládacími jednotkami alebo bezdrôtovým komunikačným modulom.

Robotika a automatizácia



Používa sa napríklad na diagnostiku, parametrizáciu alebo lokálne prepojenie riadiacej jednotky so snímačom či pomocným modulom.

Typické využitie rozhrania USART

Prenos údajov a príkazov

- merané a stavové údaje zo snímačov a modulov,
- príkazy medzi dvoma mikrokontrolérmi,
- diagnostické hlásenia a ladenie programu,
- konfigurácia zariadenia cez terminál.

Pripojenie externých zariadení

- GNSS/GPS, Bluetooth, Wi-Fi alebo GSM/LTE moduly,
- počítač cez USB–UART prevodník,
- zariadenie RS-232 cez prevodník úrovní,
- RS-485 linka cez príslušný transceiver.

Spoločný princíp

Periféria USART vytvára sériový dátový tok. Spôsob fyzického pripojenia a elektrické úrovne však určuje rozhranie zariadenia na druhej strane komunikácie.

UART

Universal Asynchronous Receiver/Transmitter

- asynchrónny sériový prenos,
- vysielač a prijímač,
- bez samostatného hodinového vodiča.

USART

Universal Synchronous/Asynchronous Receiver/Transmitter

- podporuje asynchrónny aj synchrónny režim,
- `ATmega328P` obsahuje jednu periférium `USART0`.

Asynchrónny a synchrónny prenos

Asynchrónny režim

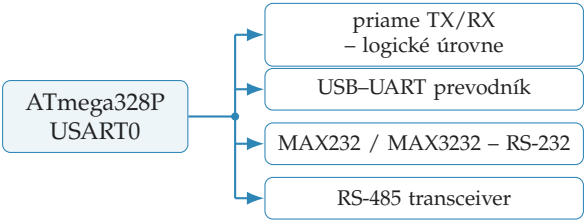
- nevysiela sa samostatný hodinový signál,
- obe strany musia poznať rovnakú prenosovú rýchlosť,
- každý rámec obsahuje štartovací a ukončovací bit,
- prijímač sa zosynchronizuje pri začiatku každého rámca.

Synchrónny režim

- používa sa hodinový signál `XCK`,
- časovanie bitov je odvodené od spoločných hodinových impulzov,
- rámec nepotrebuje rovnaký spôsob časovej synchronizácie ako asynchrónny prenos.

Princíp

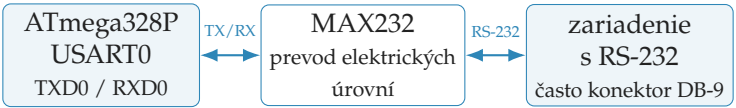
Periféria `USART` vytvára a prijíma sériový dátový tok na signáloch `TX` a `RX`. Fyzické pripojenie k druhému zariadeniu však závisí od použitého elektrického rozhrania.



Rozlíšenie

`USART` nie je to isté ako `RS-232`, `USB` alebo `RS-485`. Pri týchto rozhraniach je medzi USART a linkou potrebný príslušný prevodník alebo transceiver.

Prepojenie USART s rozhraním RS-232



Prečo je potrebný prevodník

- piny mikrokontroléra používajú logické napäťové úrovne,
- RS-232 používa kladné aj záporné napätia,
- signály RS-232 preto nemožno pripojiť priamo na `RXD0`/`TXD0`.

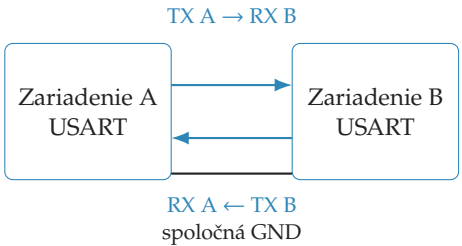
Úloha MAX232

- prevedie napäťové úrovne oboma smermi,
- zabezpečí aj potrebnú logickú inverziu,
- nemení prenášaný bajt ani formát rámca USART.

Dátové piny USART0

ATmega328P

- `PD0` / `RXD0` – vstup prijímača,
- `PD1` / `TXD0` – výstup vysielača,
- na Arduino UNO R3 zodpovedajú pinom **D0 / RX** a **D1 / TX**.



Zapojenie dvoch zariadení

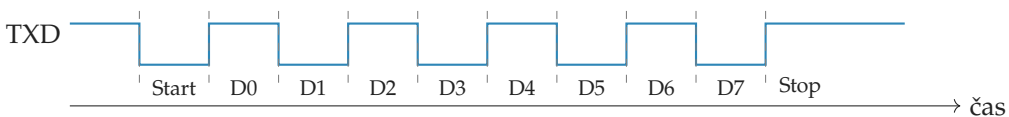
Vysielač jednej strany sa pripája na prijímač druhej strany: `TX` → `RX`, `RX` ← `TX`. Zariadenia musia mať spoločnú referenčnú zem a kompatibilné napäťové úrovne.

Asynchrónny dátový rámec

Štartovací bit, dátové bity, parita a ukončovací bit

Poradie častí rámca

1. **start bit** – prechod linky do logickej 0 označí začiatok rámca,
2. **data bits** – vlastné prenášané dáta, pri USART sa vysiela najprv bit najnižšej váhy,
3. **parity bit** – voliteľná jednoduchá kontrola prenosu,
4. **stop bit** – logická 1 ukončujúca rámec.



8N1

- **8** – osem dátových bitov,
- **N** – bez parity (*None*),
- **1** – jeden stop bit.

Celková dĺžka rámca

Pri konfigurácii 8N1 obsahuje jeden prenášaný bajt:

$$1 + 8 + 1 = 10 \text{ bitov}.$$

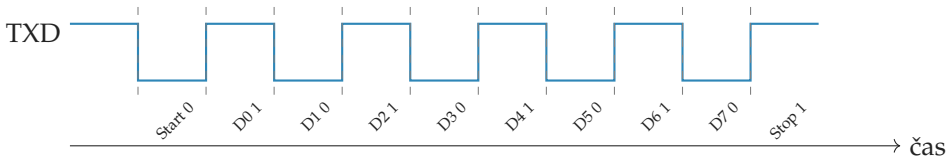
Jeden bajt používateľských dát preto neznamená iba osem bitov prenesených po linke.

Štandardné nastavenie

Budeme používať asynchrónny prenos **9600 bit/s, 8N1**.

Príklad prenosu hodnoty 0x55

Hodnota `0x55` má binárny zápis `01010101`. USART však vysiela dátové bity od najnižšej váhy `D0` po `D7`.



Interpretácia

Linka je v pokoji v logickej 1. Start bit ju stiahne do 0, potom nasleduje osem dátových bitov a stop bit v logickej 1.

Prenosová rýchlosť a čas jedného bitu

Pri asynchrónnom binárnom prenose používame prenosovú rýchlosť v bit/s. Čas jedného bitu je:

$$T_b = \frac{1}{BAUD}.$$

Pri 9600 bit/s

$$T_b = \frac{1}{9600} \approx 104.17\,\mu s$$

Jeden bit trvá približne $104\,\mu s$.

Rámec 8N1

Desať bitov na jeden bajt:

$$T_{frame} \approx 10 \cdot 104.17\,\mu s \approx 1.04\,ms.$$

Bez medzier je maximum približne 960 bajtov/s.

Ako sa prijímač zosynchronizuje bez hodinového vodiča

Princíp

Prijímač sleduje linku `RXD0`. Keď rozpozná prechod z pokojovej logickej 1 do logickej 0, začne spracovanie start bitu a podľa nastavenej prenosovej rýchlosti vzorkuje ďalšie bity rámca.

Vysielač

Používa vlastný hodinový zdroj a nastavenie prenosovej rýchlosti na určenie okamihov zmeny bitov.

Prijímač

Používa vlastný hodinový zdroj. Preto musia byť prenosová rýchlosť a presnosť hodín oboch strán dostatočne blízke.

Výpočet registra prenosovej rýchlosti

Pre asynchrónny normálny režim (`U2X0` = 0) platí:

$$BAUD = \frac{f_{CPU}}{16\,(UBRR0 + 1)}$$

a teda:

$$UBRR0 = \frac{f_{CPU}}{16\,BAUD} - 1.$$

Význam

- f_{CPU} – hodinová frekvencia mikrokontroléra,
- $BAUD$ – požadovaná prenosová rýchlosť,
- `UBRR0` – 12-bitová hodnota uložená v registroch `UBRR0H` a `UBRR0L`.

Príklad: 9600 bit/s pri 16 MHz

Pre Arduino UNO R3 s $f_{CPU} = 16\,MHz$:

$$UBRR0 = \frac{16\,000\,000}{16 \cdot 9600} - 1 \approx 103.17.$$

Do registra zapíšeme celočíselnú hodnotu:

$$\boxed{UBRR0 = 103}$$

Skutočná rýchlosť

$$BAUD_{real} = \frac{16\,000\,000}{16(103+1)} \approx 9615.4\ \text{bit/s}.$$

Odchýlka od 9600 bit/s je približne +0.16 %.

Normálny a dvojnásobný režim rýchlosti

U2X0 = 0

Normálny asynchrónny režim:

$$BAUD = \frac{f_{CPU}}{16(UBRR0 + 1)}$$

Prijímač používa vyššiu mieru prevzorkovania.

U2X0 = 1

Dvojnásobný režim:

$$BAUD = \frac{f_{CPU}}{8(UBRR0 + 1)}$$

Môže znížiť chybu pre niektoré kombinácie f_{CPU} a požadovanej rýchlosti.

Voľba režimu

Hodnotu `UBRR0` a prípadné použitie `U2X0` treba voliť podľa požadovanej rýchlosti a výslednej chyby časovania, nie automaticky. U2X0 nemení automaticky požadovanú prenosovú rýchlosť. Mení vnútorné časovanie USART a pri niektorých baud rate umožní presnejšie nastavenie prenosovej rýchlosti.

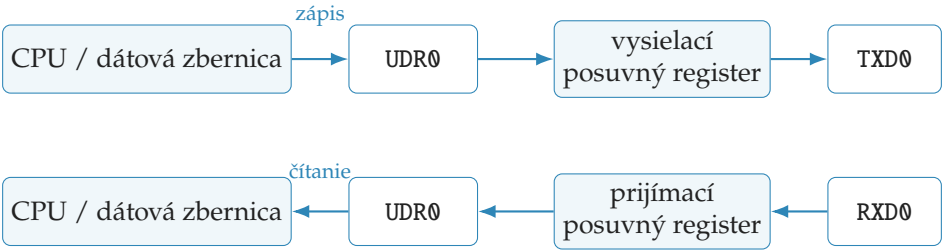
Parametre komunikácie

- prenosovú rýchlosť,
- počet dátových bitov,
- režim parity,
- počet stop bitov,
- kompatibilné elektrické úrovne a správne prepojenie vodičov.

USART0 v ATmega328P

Dátový register, stavové príznaky a konfiguračné registre

Dátová cesta periférie USART0



Plný duplex

Vysielacia a prijímacia časť pracujú nezávisle. Mikrokontrolér môže prijímať a vysielať súčasne.

USART I/O Data Register

- zápis do `UDR0` vloží dáta do vysielacieho bufferu,
- čítanie `UDR0` vráti prijaté dáta z prijímacieho bufferu,
- program pred zápisom kontroluje, či je vysielací buffer pripravený,
- program pred čítaním kontroluje, či prijímač obsahuje neprečítané dáta.

CPU → `UDR0` → vysielač

prijímač → `UDR0` → CPU

Stavový register UCSR0A

Bit	Názov	Význam
7	RXC0	prijaté neprečítané dáta
6	TXC0	vyslaný celý rámec
5	UDRE0	dátový register vysielača je pripravený
4	FE0	chyba rámca
3	DOR0	pretečenie prijímača
2	UPE0	chyba parity
1	U2X0	dvojnásobná prenosová rýchlosť

Najčastejšie používané príznaky

Pri blokujúcom odosielaní sledujeme `UDRE0`. Pri blokujúcom príjme sledujeme `RXC0`.

Rozdiel medzi UDRE0 a TXC0

UDRE0

USART Data Register Empty

- vysielací dátový register môže prijať ďalší bajt,
- neznamená, že predchádzajúci rámec už opustil pin `TXD0`,
- používa sa pri posielaní ďalších bajtov za sebou.

TXC0

USART Transmit Complete

- celý rámec bol vysunutý z posuvného registra,
- vysielač už nemá čakajúce dáta,
- dôležité napríklad pred vypnutím vysielača alebo zmenou smeru poloduplexnej linky.

Riadiaci register UCSR0B

Bit	Názov	Význam
7	RXCIE0	prerušenie po prijatí bajtu
6	TXCIE0	prerušenie po úplnom odvysielaní rámca
5	UDRIE0	prerušenie pri pripravenosti dátového registra
4	RXEN0	povolenie prijímača
3	TXEN0	povolenie vysielača
2	UCSZ02	tretí bit nastavenia dĺžky znaku

Základná konfigurácia

Pre bežný 8-bitový prenos potrebujeme minimálne povoliť `RXEN0` a `TXEN0`. Prerušenia zapíname iba vtedy, keď ich program skutočne používa.

Vektor	Povolenie	Kedy sa používa
USART_RX_vect	RXCIE0	prijímač prijal nový bajt
USART_UDRE_vect	UDRIE0	vysielací dátový register môže prijať ďalší bajt
USART_TX_vect	TXCIE0	vyslanie celého rámca bolo dokončené

Rozdiel

UDRIE0 a TXCIE0 neznamenajú to isté. Prvé prerušenie slúži na priebežné dopĺňanie vysielacích dát, druhé informuje o úplnom dokončení prenosu rámca.

Riadiaci register UCSR0C

`Bity`	Funkcia	Pre konfiguráciu 8N1
`UMSEL01:0`	režim USART	`00` – asynchrónny USART
`UPM01:0`	parita	`00` – parita vypnutá
`USBS0`	počet stop bitov	`0` – jeden stop bit
`UCSZ01:0`	počet dátových bitov	11 spolu s `UCSZ02=0` – 8 bitov

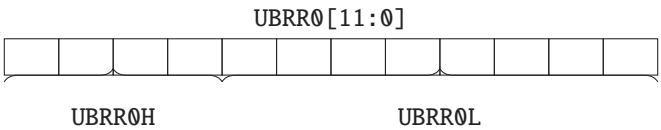
Výsledok

Pri zápise `UCSR0C = (1 « UCSZ01) | (1 « UCSZ00);` nastavíme osem dátových bitov a ostatné uvedené voľby ostanú nulové.

USART Baud Rate Register

`UBRR0` je 12-bitová hodnota rozdelená do dvoch 8-bitových registrov:

- `UBRR0H` obsahuje horné bity hodnoty,
- `UBRR0L` obsahuje dolných osem bitov.



Inicializácia USART0: 9600, 8N1

```
#define F_CPU 16000000UL
#define BAUD  9600UL
#define UBRR_VALUE ((F_CPU / (16UL * BAUD)) - 1UL)

void usart0_init(void)
{
    UBRR0H = (uint8_t)(UBRR_VALUE >> 8);
    UBRR0L = (uint8_t)UBRR_VALUE;

    UCSR0A = 0;
    UCSR0B = (1 << RXEN0) | (1 << TXEN0);
    UCSR0C = (1 << UCSZ01) | (1 << UCSZ00);
}
```

Interpretácia inicializácie USART0

Čo program nastaví

- `UBRR0H/L` – prenosová rýchlosť 9600 bit/s,
- `U2X0 = 0` – normálny asynchrónny režim,
- `RXEN0 = 1` – povolenie prijímača,
- `TXEN0 = 1` – povolenie vysielača,
- `UCSZ01:0 = 11` a `UCSZ02 = 0` – 8 dátových bitov,
- parita vypnutá a jeden stop bit.

Piny

Po povolení prijímača a vysielača periféria USART prevezme funkciu pinov `PD0/RXD0` a `PD1/TXD0`. Nie je potrebné nastavovať smer týchto pinov cez `DDRD` ako pri obyčajnom GPIO.

Prvky jazyka C pre textovú komunikáciu

Znaky, reťazce, polia a ukazovatele

Znak ako číselná hodnota

V jazyku C je typ `char` celočíselný typ vhodný na reprezentáciu znaku.

```
1 char znak = 'A';
2 uint8_t data = (uint8_t)znak;
```

ASCII

Pre základné textové príkazy používame znaky z ASCII tabuľky. Napríklad znak `'A'` má kód 65, teda `0x41`.

Znak nie je číslica

`'1'` je znak s ASCII kódom 49. Číselná hodnota 1 je iný údaj.

Znak a reťazec nie sú to isté

Jeden znak

```
1 char c = 'A';
```

Apostrofy označujú jeden znak.

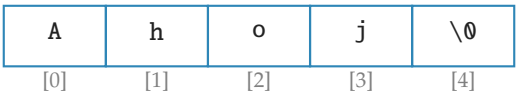
Reťazec znakov

```
1 char text[] = "Ahoj";
```

Úvodzovky označujú reťazec.

Pamäťová reprezentácia

Reťazec `Ahoj"` obsahuje päť prvkov: `'A'`, `'h'`, `'o'`, `'j'` a ukončovací znak `'\0'`.



Ukončenie reťazca

Znak `'\0'` má číselnú hodnotu 0. Funkcia, ktorá nevie dĺžku reťazca vopred, môže postupovať po poli dovtedy, kým nenájde túto nulovú hodnotu.

Ukazovateľ a adresa premennej

```
1 char znak = 'A';
2 char *p = &znak;
```

Operátor &

&znak znamená adresu premennej znak v pamäti.

Operátor *

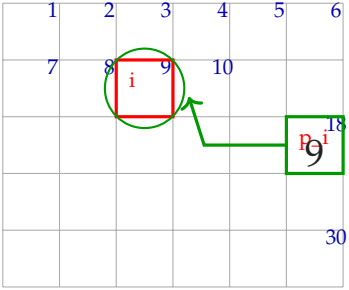
*p znamená hodnotu uloženú na adrese, ktorú obsahuje ukazovateľ p.

Interpretácia

Ak p = &znak, potom výraz *p predstavuje tú istú hodnotu ako premenná znak.

Smerníky (ukazovatele) – angl. pointers

```
1 int *p_i, i;
```

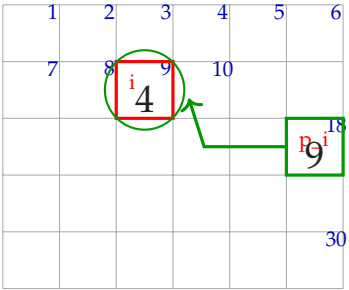


Význam

- `i` je premenná uložená na adrese 9.
- `p_i = &i;` uloží adresu premennej `i` do smerníka `p_i`.
- Hodnota smerníka `p_i` je teda 9.
- `p_i` ukazuje na bunku pamäte s adresou 9.

Prístup k hodnote cez ukazovateľ

```
1 p_i = &i;
2 *p_i = 4;
```



Význam

- `p_i` stále obsahuje adresu 9.
- Operátor * pristupuje k hodnote uloženej na tejto adrese.
- `*p_i = 4;` zapíše hodnotu 4 do bunky s adresou 9.
- Keďže na adrese 9 je premenná `i`, výsledok je rovnaký ako `i = 4;`.

Prečo používať const char *

Funkcia na odoslanie reťazca má reťazec iba čítať, nie meniť.

```
1 void usart0_send_string(const char *text);
```

Význam const

`const char *text` znamená ukazovateľ na znaky, ktoré funkcia prostredníctvom tohto ukazovateľa nebude meniť.

Výhoda

Rozhranie funkcie priamo vyjadruje jej zámer a umožňuje bezpečne odovzdať napríklad reťazec `Ahoj"`.

Odoslanie celého reťazca

```
1 void usart0_send_string(const char *text)
2 {
3     while (*text != '\0') {
4         usart0_send_byte((uint8_t)*text);
5         text++;
6     }
7 }
```

Postup

1. `*text` načíta aktuálny znak,
2. znak sa odošle cez USART,
3. `text++` posunie ukazovateľ,
4. cyklus skončí pri `'\0'`.

Pole ako prijímací buffer

Pre viacznakový príkaz potrebujeme miesto, do ktorého sa prijaté znaky postupne ukladajú.

```
1 #define RX_BUFFER_SIZE 16
2
3 char rx_buffer[RX_BUFFER_SIZE];
4 uint8_t rx_index = 0;
```

Princíp

- každý prijatý znak sa uloží na pozíciu `rx_index`,
- index sa zvýši,
- na konci textu sa vloží `'\0'`,
- program musí kontrolovať hranicu poľa, aby nezapisoval mimo bufferu.

Implementácia komunikácie USART0

Odosielanie, príjem, prerušenia a organizácia programu

Odoslanie jedného bajtu

```
1 void usart0_send_byte(uint8_t data)
2 {
3     while (!(UCSR0A & (1 << UDRE0))) {
4         // cakanie na volny datovy register
5     }
6
7     UDR0 = data;
8 }
```

Postup

1. testuje sa príznak `UDRE0`,
2. kým vysielací buffer nie je pripravený, funkcia čaká,
3. po uvoľnení sa bajt zapíše do `UDR0`.

Vlastnosť

Ide o **blokujúcu funkciu**: procesor môže počas čakania vykonávať iba tento cyklus.

Príjem jedného bajtu

```
1 uint8_t usart0_receive_byte(void)
2 {
3     while (!(UCSR0A & (1 << RXC0))) {
4         // cakanie na prijate data
5     }
6
7     return UDR0;
8 }
```

Postup

1. príznak `RXC0` signalizuje neprečítané dáta,
2. funkcia čaká na prijatie bajtu,
3. čítanie `UDR0` vráti prijatú hodnotu.

Dôsledok

Ak neprídu žiadne dáta, hlavný program zostane vo funkcii `usart0_receive_byte()`.

Príklad: spätné odoslanie prijatého znaku

```c
#include <avr/io.h>
#include <stdint.h>

int main(void)
{
    usart0_init();

    while (1) {
        uint8_t data = usart0_receive_byte();
        usart0_send_byte(data);
    }
}
```

Echo

Každý bajt prijatý z počítača sa okamžite odošle späť. Funkciu možno jednoducho overiť v sériovom termináli.

Príklad: ovládanie LED textovým príkazom

```
DDRB |= (1 << PB5);

while (1) {
    char c = (char)usart0_receive_byte();

    if (c == '1') {
        PORTB |= (1 << PB5);
        usart0_send_string("LED ON\r\n");
    }
    else if (c == '0') {
        PORTB &= ~(1 << PB5);
        usart0_send_string("LED OFF\r\n");
    }
}
```

Prepojenie tém

- USART prijme znak,
- podmienka porovná znak s `'1'` alebo `'0'`,
- GPIO zmení stav LED,
- USART odošle textovú odpoveď.

Znaky \r a \n v termináli

'\r'

Carriage Return – návrat kurzora na začiatok riadku.

'\n'

Line Feed – posun na nový riadok.

Bežný textový výstup

Pre terminál je praktické posielať koniec riadku ako `"\r\n"`, napríklad `"LED ON\r\n"`.

Príjem pomocou prerušenia

```
1 volatile uint8_t rx_data = 0;
2 volatile uint8_t rx_ready = 0;
3
4 ISR(USART_RX_vect)
5 {
6     rx_data = UDR0;
7     rx_ready = 1;
8 }
```

Potrebné nastavenie

- `RXCIE0 = 1`,
- globálne povolenie prerušení pomocou `sei()`,
- ISR má zostať krátka,
- hlavný program spracuje prijatý bajt až mimo ISR.

Obmedzenie jednoduchého príkladu

Ak príde ďalší bajt skôr, než hlavný program spracuje predchádzajúci, jedna premenná `rx_data` nestačí. V reálnom programe sa používa prijímací buffer.

Chybové príznaky pri príjme

```
uint8_t status = UCSR0A;
uint8_t data   = UDR0;

if (status & ((1 << FE0) | (1 << DOR0) | (1 << UPE0))) {
    // spracovanie chyby prijmu
}
```

Význam príznakov

- `FE0` – chyba rámca, napríklad neplatný stop bit,
- `DOR0` – nové dáta prišli skôr, než sa stihli spracovať predchádzajúce,
- `UPE0` – chyba parity, ak je parita povolená.

Od jedného bajtu k prijímaciemu bufferu

Prečo buffer

USART môže prijímať dáta nezávisle od toho, čo práve robí hlavný program. Buffer oddelí okamih prijatia bajtu od okamihu jeho spracovania.

ISR

- rýchlo prečíta `UDR0`,
- uloží bajt do poľa,
- aktualizuje index alebo pozíciu v kruhovom bufferi.

Hlavná slučka

- zisťuje, či sú v buffri dáta,
- spracuje príkaz alebo text,
- vykoná časovo náročnejšie operácie mimo ISR.

Organizácia programu v jazyku C

Vlastné .h/.c súbory, preprocesor, samostatný preklad a linkovanie

Úloha hlavičkového a implementačného súboru

`usart0.h`

Opisuje **verejné rozhranie** modulu:

- funkčné prototypy,
- dátové typy a makrá používané mimo modulu,
- ochranu proti viacnásobnému vloženiu.

`usart0.c`

Obsahuje **implementáciu**:

- definície funkcií,
- prácu s registrami USART0,
- interné pomocné premenné a funkcie.

Pravidlo

Súbor `main.c` vkladá `usart0.h`, nie `usart0.c`. Hlavičkový súbor sa neprekladá samostatne. Jednotlivé súbory `.c` sa prekladajú samostatne a spoja sa až pri linkovaní.

Čo vykoná direktíva #include

```c
/* main.c */
#include "usart0.h"

int main(void)
{
    usart0_init();
    usart0_send_string("Ahoj\r\n");

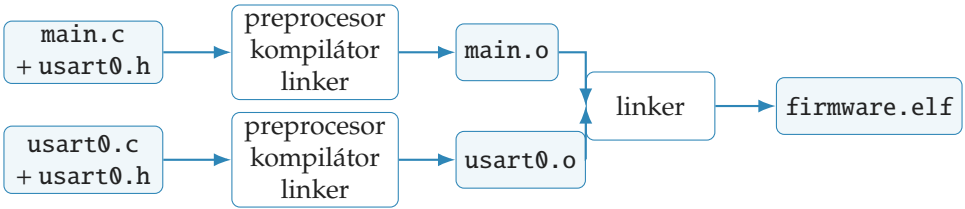
    while (1) { }
}
```

Preprocesor

Direktíva `#include "usart0.h"` spôsobí, že preprocesor vloží obsah hlavičkového súboru do spracovávaného zdrojového súboru ešte pred samotným prekladom.

Prečo je prototyp potrebný

Kompilátor pri preklade `main.c` pozná názov funkcie, typ návratovej hodnoty a jej parametre, aj keď jej telo je definované v inom súbore.



`firmware.elf` → `avr-objcopy` → `firmware.hex`

Postup

Každý súbor `.c` tvorí samostatnú prekladovú jednotku. Po preklade vzniknú objektové súbory `.o`. Linker ich spojí, priradí volania funkcií k ich definíciám a vytvorí výsledný program `.elf`. Z neho sa následne vytvorí súbor určený na zápis do pamäte Flash mikrokontroléra.

Prepojenie main.c a usart0.c

```c
/* usart0.h */
#ifndef USART0_H
#define USART0_H

void usart0_init(void);
void usart0_send_string(const char *text);

#endif
```

Pri preklade

- `main.c` pozná funkcie z prototypov v `usart0.h`,
- `usart0.c` obsahuje ich skutočné definície,
- linker nájde príslušné definície a spojí ich s volaniami z `main.c`.

Dôležité

Hlavičkový súbor sa bežne vkladá do `main.c` aj do príslušného `usart0.c`. Tým sa kontroluje, že deklarácie verejných funkcií zodpovedajú ich implementácii.

Hlavičkový súbor usart0.h

```
#ifndef USART0_H
#define USART0_H

#include <stdint.h>

void usart0_init(void);
void usart0_send_byte(uint8_t data);
uint8_t usart0_receive_byte(void);
void usart0_send_string(const char *text);

#endif
```

Poznámka

Pri bežných funkčných prototypoch nie je potrebné písať kľúčové slovo `extern`; funkcie majú štandardne externú väzbu, pokiaľ nie sú deklarované ako `static`.

Implementačný súbor usart0.c

```c
#define F_CPU 16000000UL

#include <avr/io.h>
#include "usart0.h"
#define BAUD  9600UL
#define UBRR_VALUE ((F_CPU / (16UL * BAUD)) - 1UL)

void usart0_init(void)
{
    UBRR0H = (uint8_t)(UBRR_VALUE >> 8);
    UBRR0L = (uint8_t)UBRR_VALUE;
    UCSR0A = 0;
    UCSR0B = (1 << RXEN0) | (1 << TXEN0);
    UCSR0C = (1 << UCSZ01) | (1 << UCSZ00);
}
```

Implementácia odosielania v usart0.c

```
1  void usart0_send_byte(uint8_t data)
2  {
3      while (!(UCSR0A & (1 << UDRE0))) { }
4      UDR0 = data;
5  }
6
7  void usart0_send_string(const char *text)
8  {
9      while (*text != '\0') {
10         usart0_send_byte((uint8_t)*text);
11         text++;
12     }
13 }
```

Hlavný program s modulom USART

```c
#include <avr/io.h>
#include "usart0.h"

int main(void)
{
    usart0_init();
    usart0_send_string("USART pripraveny\r\n");

    while (1) {
        uint8_t data = usart0_receive_byte();
        usart0_send_byte(data);
    }
}
```

Výhoda rozdelenia

Súbor `main.c` používa USART cez jasné rozhranie a nemusí obsahovať detaily nastavovania registrov periférie.

Nastavenie terminálu

- zvoliť správny sériový port,
- nastaviť 9600 bit/s, 8 dátových bitov, bez parity, 1 stop bit,
- odoslať znaky a sledovať odpoveď mikrokontroléra.

Port v operačnom systéme

Vo Windows sa port typicky zobrazuje ako `COMx`; v Linuxe môže UNO R3 vystupovať napríklad ako `/dev/ttyACM0`.

Kľúčové body

- USART0 na ATmega328P umožňuje asynchrónny aj synchrónny sériový prenos.
- Pri asynchrónnom prenose musia mať obe strany zhodnú prenosovú rýchlosť a formát rámca.
- Pre konfiguráciu 9600, 8N1 používame najmä registre `UBRR0H/L`, `UCSR0A/B/C` a `UDR0`.
- `UDRE0` signalizuje pripravenosť vysielacieho dátového registra, `RXC0` prítomnosť prijatých dát.
- Textová komunikácia v C vyžaduje prácu s typom `char`, poľami, nulovým znakom a ukazovateľmi.
- Blokujúci prístup je jednoduchý, prerušenia a buffre umožňujú lepšie oddeliť komunikáciu od hlavnej činnosti programu.

Ďakujem za pozornosť